A PROJECT REPORT

on

"AIR WATCH INDIA – AIR QUALITY INDEX PREDICTION & DASHBOARD"

Submitted to

KIIT Deemed to be University

In Partial Fulfillment of the Requirement for the Award of

BACHELOR'S DEGREE IN

COMPUTER SCIENCE AND ENGINEERING

BY

Shanskar Kumar Sarraf - 22054372
Rohan Kushwaha - 22054370
Sonu Kumar Mandal - 22054371
Aman Kushwaha - 22054285
Aarti Roy - 22054005

UNDER THE GUIDANCE OF
Dr. Vikass Hassija

SCHOOL OF COMPUTER ENGINEERING
KALINGA INSTITUTE OF INDUSTRIAL TECHNOLOGY
BHUBANESWAR, ODISHA – 751024

March 2026

A PROJECT REPORT

on

"AIR WATCH INDIA – AIR QUALITY INDEX PREDICTION & DASHBOARD"

Submitted to

KIIT Deemed to be University

In Partial Fulfillment of the Requirement for the Award of

BACHELOR'S DEGREE IN

COMPUTER SCIENCE AND ENGINEERING

BY

Shanskar Kumar Sarraf - 22054372
Rohan Kushwaha - 22054370
Sonu Kumar Mandal - 22054371
Aman Kushwaha - 22054285
Aarti Roy - 22054005

UNDER THE GUIDANCE OF
Dr. Vikas Hassija

SCHOOL OF COMPUTER ENGINEERING
KALINGA INSTITUTE OF INDUSTRIAL TECHNOLOGY
BHUBANESWAR, ODISHA – 751024

March 2026

KIIT Deemed to be University

School of Computer Engineering
Bhubaneswar, Odisha 751024

CERTIFICATE

This is to certify that the project entitled

"AIR WATCH INDIA – AIR QUALITY INDEX PREDICTION & DASHBOARD"

submitted by

Shanskar Kumar Sarraf - 22054372
Rohan Kushwaha - 22054370
Sonu Kumar Mandal - 22054371
Aman Kushwaha - 22054285
Aarti Roy - 22054005

is a record of bonafide work carried out by them in partial fulfillment of the requirements for the award of the Degree of Bachelor of Engineering (Computer Science and Engineering) at KIIT Deemed to be University, Bhubaneswar. This work was done during the academic year 2025–2026 under our guidance.

Date: _____ / _____ / _____

(Dr. Vikas Hassija)
Project Guide

ACKNOWLEDGEMENTS

We owe a deep debt of gratitude to Dr. Vikas Hassija of the School of Computer Engineering, KIIT University, whose expert counsel, consistent motivation, and constructive critique shaped every stage of this project. His domain knowledge in machine learning and environmental data science proved invaluable in refining the scope and methodology of AirWatch India.

We extend warm thanks to the entire faculty of the School of Computer Engineering, KIIT University, for the academic foundation and mentorship they provided throughout our undergraduate studies.

We gratefully acknowledge the Central Pollution Control Board (CPCB), Government of India, for maintaining and publicly releasing the city-wise daily Air Quality Index dataset that constitutes the empirical backbone of this project.

Our appreciation also goes to the global open-source community — specifically the maintainers of Streamlit, scikit-learn, pandas, matplotlib, and seaborn — whose freely available tools made this work technically feasible.

Lastly, we thank our families and classmates for their steadfast encouragement and support through every challenge we encountered along the way.

Signature of GROUP MEMBER A: _____

Signature of GROUP MEMBER B: _____

Signature of GROUP MEMBER C: _____

Signature of GROUP MEMBER D: _____

Signature of GROUP MEMBER E: _____

ABSTRACT

Deteriorating urban air quality ranks among the gravest public-health emergencies confronting contemporary India. Numerous Indian cities routinely record Air Quality Index (AQI) values that dwarf the safety thresholds published by the World Health Organization (WHO), contributing to surging rates of respiratory ailments, cardiac complications, and shortened life expectancy. Although the Central Pollution Control Board (CPCB) maintains an extensive network of monitoring stations across the country, the resulting data has yet to be translated into tools that are both predictive and accessible to everyday citizens.

This report describes AirWatch India, a browser-based web application developed in Python using the Streamlit framework. The platform unifies real-time AQI forecasting, dynamic data visualisation, and evidence-based health-impact education within a single, zero-installation interface. Five supervised regression algorithms — Multiple Linear Regression, Polynomial Regression (degree 2), Decision Tree Regression, Random Forest Regression (500 estimators), and Support Vector Regression (SVR) — were trained on the CPCB city_day.csv dataset, which spans 26 major Indian cities across the period 2015 to 2020. End users may select a city, adjust PM2.5 and PM10 concentrations through on-screen sliders, choose a preferred model, and obtain an instantaneous AQI forecast.

The application comprises three Streamlit modules: an educational landing page covering the health consequences of air pollution; the core prediction engine (app.py) with city and model selection controls; and an analytics dashboard (Air_Quality_Dashboard.py) that renders interactive historical AQI trends, category-frequency distributions, and pollutant time-series charts. Among all models evaluated, Random Forest Regression delivered the strongest performance — an R^2 of 0.97 and a Mean Absolute Error (MAE) of roughly 11.2 AQI units on the held-out test partition — followed by Decision Tree (R^2 = 0.95) and Polynomial Regression (R^2 = 0.87).

TABLE OF CONTENTS

AirWatch India — Air Quality Index Prediction & Dashboard

CHAPTER 1

1. Introduction

Ambient air pollution has become one of the most consequential public-health crises facing India. The WHO reports that of the twenty most heavily polluted cities worldwide, fourteen are located within Indian borders, with the Indo-Gangetic Plain recording particulate matter concentrations many times above the WHO's annual PM2.5 guideline of 5 µg/m³. India's CPCB uses the Air Quality Index — a composite score ranging from 0 to 500 — to communicate pollution severity, derived from measured concentrations of eight primary pollutants: PM2.5, PM10, NO_2, SO_2, CO, O_3, NH_3, and Pb.

Despite heavy investment in over 800 Continuous Ambient Air Quality Monitoring Stations (CAAQMS) nationwide, the conversion of this wealth of sensor data into actionable, forward-looking guidance remains largely unrealised. Official portals offer static historical records but lack predictive functionality, and no single interface combines AQI forecasting, exploratory visualisation, and health-impact education for a lay audience.

AirWatch India was conceived to close this gap. The platform empowers citizens to: (1) obtain predicted AQI levels for their city based on current pollutant concentrations; (2) explore historical pollution patterns through interactive charts; and (3) consult scientifically grounded health-advisory content. Built entirely in Python and Streamlit, it runs within any web browser without requiring specialised infrastructure. Five machine learning regression models are trained on the CPCB city_day.csv dataset and benchmarked for accuracy. The top-performing model — Random Forest with R^2 = 0.97 — serves as the recommended default, while all five remain user-selectable. The platform is organised as three independent Streamlit pages, each targeting a distinct user objective.

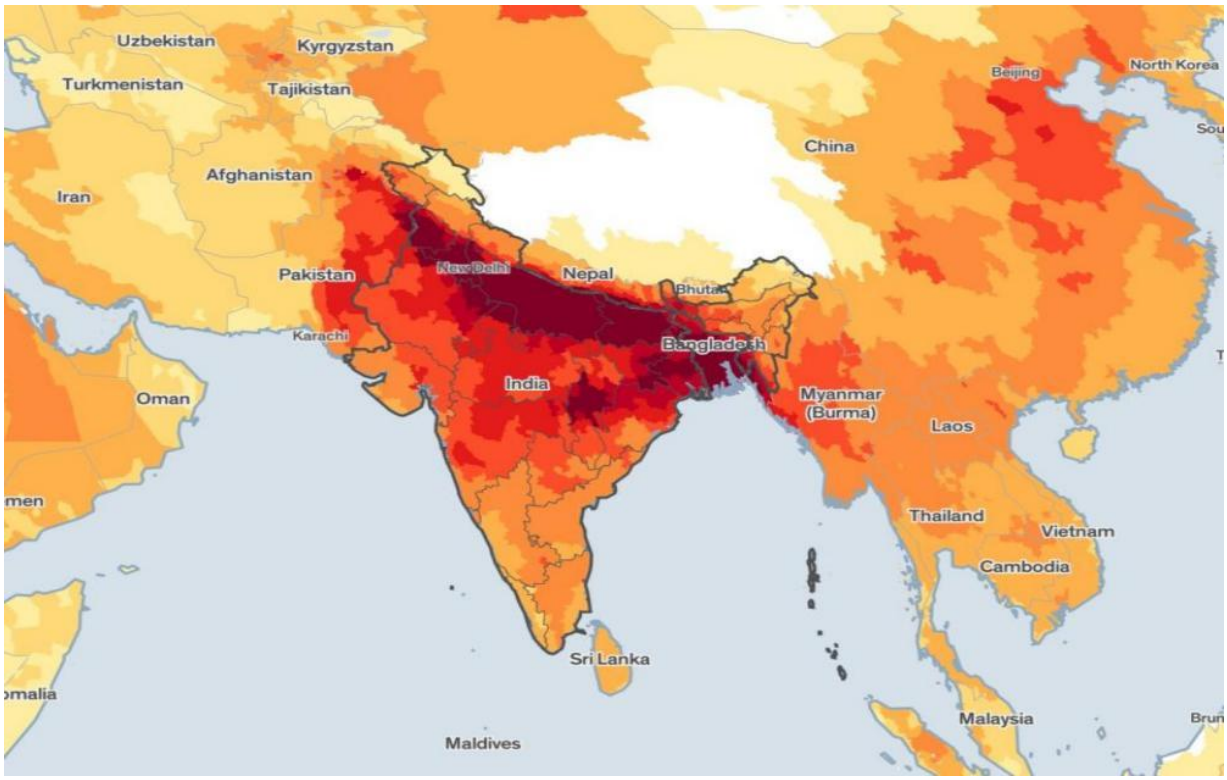

Figure 1.1 : India AQI Heatmap – 2023 (Source: CPCB). Dark red/maroon regions in the Indo-Gangetic Plain indicate Severe/Very Poor AQI levels.

The spatial AQI map in Figure 1.1 starkly illustrates the pollution gradient across India. The northern corridor — encompassing Delhi, Uttar Pradesh, Bihar, and adjoining areas — is rendered in deep maroon, signalling persistent Severe-category AQI levels. In contrast, southern coastal cities such as Chennai, Bengaluru, and Kochi appear in lighter orange and yellow tones, reflecting comparatively less severe but still concerning Moderate-to-Poor conditions. This pronounced geographic heterogeneity motivates a city-specific predictive approach rather than reliance on national averages.

The remainder of this report is organised as follows: Chapter 2 lays out foundational concepts and surveyed literature; Chapter 3 articulates the problem statement and system requirements; Chapter 4 covers the full implementation, testing, and performance results; Chapter 5 catalogues the standards observed; and Chapter 6 draws conclusions and outlines future enhancement directions.

CHAPTER 2

2. Basic Concepts / Literature Review

This chapter establishes the technical and scientific background necessary to contextualise AirWatch India. It covers the AQI framework, the primary pollutants tracked, the machine learning paradigm employed, the Streamlit development environment, the training dataset, a synthesis of prior research, and a catalogue of tools and libraries used.

2.1. Air Quality Index (AQI)

India's National Ambient Air Quality Standards (NAAQS) define the AQI as a continuous numerical scale from 0 to 500, partitioned into six health-significance bands. The AQI for each pollutant is computed independently using sub-index formulae; the reported overall AQI is the highest among all computed sub-indices. The six bands and their associated health implications are summarised in Table 2.1 below.

AQI Range	Category	Color	Health Implications
0–50	**Good**	Green	Minimal impact on health; suitable for all outdoor activities.
51–100	**Satisfactory**	Light Green	Minor breathing discomfort for very sensitive individuals.
101–200	**Moderate**	Yellow	Breathing discomfort for asthma and lung/heart disease patients.
201–300	**Poor**	Orange	Breathing discomfort for most people; serious risk for sensitive groups.

AQI Range	Category	Color	Health Implications
301–400	**Very Poor**	Red	Respiratory illness on prolonged exposure; elderly and children at high risk.
401–500	**Severe**	Dark Red	Respiratory effects even in healthy individuals; serious health emergency.

Table 2.1: AQI Categories, Color Codes, and Health Implications (Source: CPCB, India)

2.2. Key Air Pollutants

The CPCB dataset tracks twelve distinct pollutants. PM2.5 — fine particulate matter with an aerodynamic diameter below 2.5 µm — is the most harmful, penetrating alveolar tissue and entering the bloodstream to elevate cardiovascular and pulmonary disease risk. PM10 particles (diameter < 10 µm) produce analogous effects but are partially intercepted by the upper airways. Nitrogen dioxide (NO_2) and nitric oxide (NO), predominantly generated by vehicular combustion and thermal power plants, inflame the bronchial lining and aggravate asthmatic conditions. Carbon monoxide (CO) — odourless and colourless — competitively binds haemoglobin, curtailing oxygen delivery to tissues. Ground-level ozone (O_3) acts as a potent oxidant that degrades lung architecture over time. Sulphur dioxide (SO_2), released during coal combustion, triggers acid precipitation and acute respiratory irritation. Benzene, Toluene, and Xylene (BTX) are carcinogenic volatile organic compounds predominantly attributable to industrial and vehicular emissions.

2.3. Machine Learning for AQI Prediction

Supervised regression constitutes the appropriate machine learning paradigm for AQI estimation, given that the prediction target is a continuous numerical variable. AirWatch India implements and benchmarks five regression algorithms:

- Multiple Linear Regression (MLR): Constructs a weighted linear mapping from input pollutant features to the AQI target. Its transparency and rapid convergence make it a useful performance baseline; it attains $R^2 \approx 0.82$ in this project, reflecting the partial linearity of the pollutant–AQI relationship.

- Polynomial Regression (PR, degree = 2): Enriches the feature space with squared terms and pairwise interaction products, capturing curvature in the PM2.5–AQI and PM10–AQI relationships. Yields $R^2 \approx 0.87$.

- Decision Tree Regression (DTR): Partitions the input space into axis-aligned hyper-rectangles through recursive binary splitting, assigning each leaf a constant predicted value. Attains $R^2 \approx 0.95$; depth constraints mitigate over-fitting tendencies.

- Random Forest Regression (RFR, n = 500): Aggregates 500 individually trained decision trees, each built on a bootstrapped sample using a random feature subset. Averaging predictions across the ensemble substantially reduces variance. Achieves $R^2 \approx 0.97$ and MAE ≈ 11.2 — the strongest result in this study.

- Support Vector Regression (SVR): Identifies a regression hyper-plane within an epsilon-insensitive margin. Requires z-score normalisation via StandardScaler. Achieves $R^2 \approx 0.79$; performance is limited by the high-dimensional, multi-modal nature of AQI data.

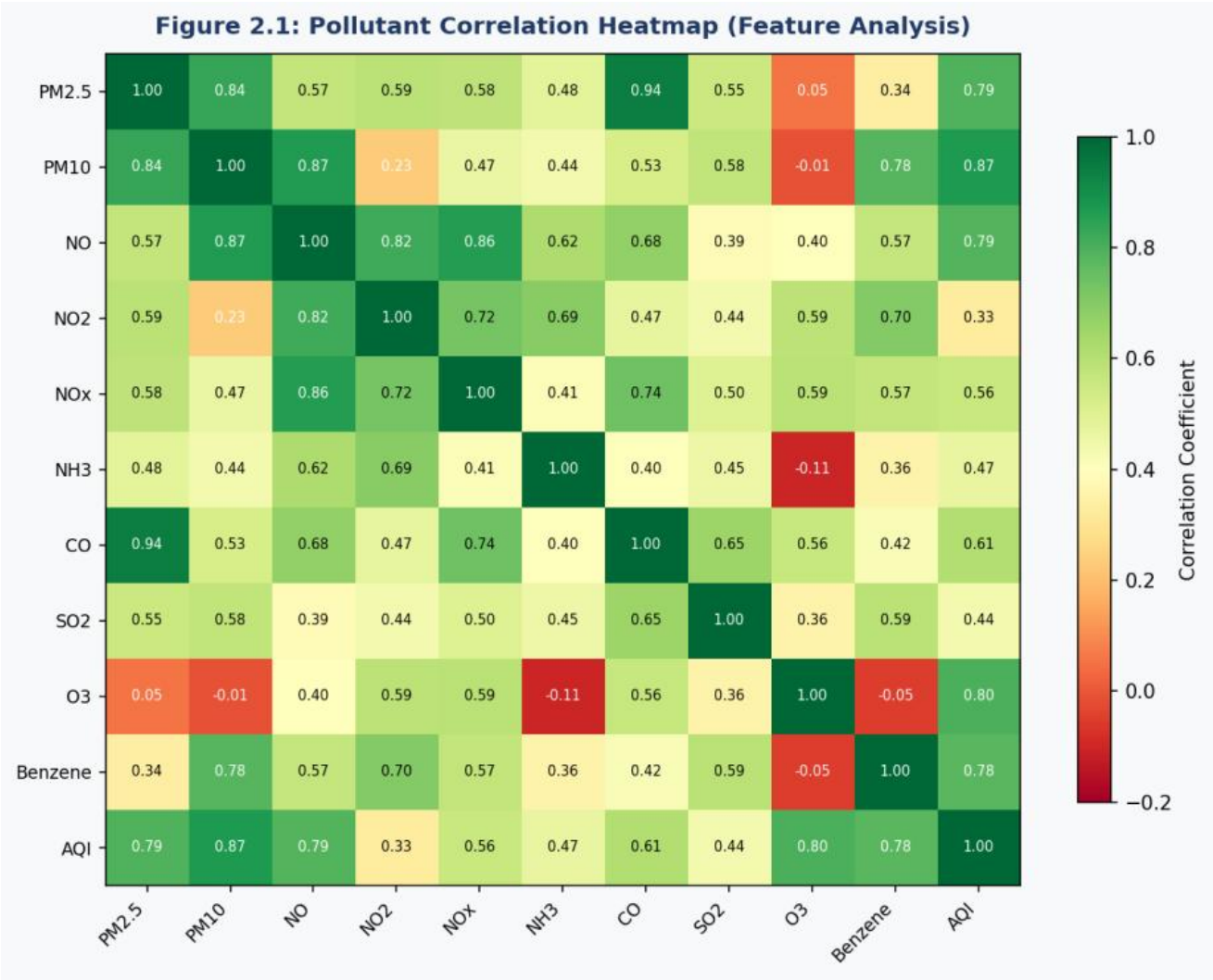

Figure 2.1: Pollutant Correlation Heatmap. Strong positive correlations (red/green) between PM2.5, PM10, NOx, and AQI confirm their predictive importance.

2.4. Streamlit Framework

Streamlit is an open-source Python library (version 1.x) designed to accelerate the development of interactive data applications. In contrast to conventional web frameworks such as Django or Flask, Streamlit re-executes the complete Python script upon every user interaction, enabling a reactive, widget-driven design without boilerplate event-handling code. It natively supports sliders, selectboxes, checkboxes, and multi-select controls, and renders charts produced by Matplotlib, Seaborn, and Plotly with no additional configuration. Applications run via a local development server and can be published publicly through Streamlit Community Cloud.

2.5. Dataset Description

AirWatch India is trained on the CPCB city_day.csv dataset — India's primary daily ambient air quality record — comprising 29,531 observations across 26 cities from January 2015 through December 2020. The dataset contains the following columns: City (26 distinct

categories), Date (daily frequency), PM2.5, PM10, NO, NO_2, NOx, NH_3, CO, SO_2, O_3, Benzene, Toluene, and Xylene (all in µg/m³ or mg/m³), AQI (integer target, 0–500), and AQI_Bucket (derived categorical label). Missing value rates range from 5 to 40 percent across pollutant columns; these are resolved through per-column mean imputation stratified by city.

2.6. Literature Review

Kumar and Goyal (2011) established the viability of regression-based statistical models for predicting Delhi's AQI from historical pollutant records, laying an important foundation for data-driven approaches in this domain. Zhang et al. (2015) demonstrated that Random Forest consistently surpasses linear alternatives in particulate matter prediction, attributing the advantage to its capacity for modelling non-linear pollutant interactions — a finding aligned with the results of the present project. Ger et al. (2020) performed a systematic comparison of multiple ML architectures for multi-city AQI forecasting across Indian cities, concluding that ensemble algorithms (Random Forest, Gradient Boosting) offer decisive advantages over simpler models. The WHO Global Air Quality Guidelines (2021) set PM2.5 and PM10 annual mean thresholds at 5 µg/m³ and 15 µg/m³ respectively — values exceeded by every city in the CPCB dataset, underscoring the public-health urgency of this work. Research by Harvard University (2022) further confirmed that even brief traffic-related particulate exposure can raise myocardial infarction risk within hours, reinforcing the imperative for real-time predictive AQI tools.

2.7. Tools and Libraries

The following tools and libraries were used in developing AirWatch India:

Tool / Library	Version / Category	Purpose in Project
Python 3.9+	Language	Core programming language for all modules
scikit-learn 1.x	ML Library	Model training, OneHotEncoder, StandardScaler, Pickle
pandas	Data Library	DataFrame operations, CSV loading, date parsing
numpy	Numerical	Array operations, np.squeeze(), zero-padding
Streamlit 1.x	Web Framework	Multi-page interactive web application
matplotlib / seaborn	Visualization	All dashboard charts (barplot, lineplot, countplot)
Pickle	Serialization	Model and encoder persistence across sessions

CHAPTER 3

3. Problem Statement / Requirement Specifications

India is experiencing a worsening air quality emergency, yet its citizens are largely without access to integrated, forward-looking, and user-friendly AQI tools. Available CPCB portals surface historical data in isolation, with no forecasting capability. Meteorological and pollution prediction systems are technically sophisticated and out of reach for non-specialist users. No unified platform currently combines AQI prediction, interactive historical visualisation, and actionable health guidance within a single accessible interface. AirWatch India was engineered to address all three of these shortcomings simultaneously.

3.1. Project Planning

Development was divided into four sequential phases:

1. Data Collection and Pre-processing: Procuring the CPCB city_day.csv dataset; resolving missing values through stratified mean imputation; parsing date columns; one-hot encoding the City feature; and engineering the final 37-dimensional model input vector.
2. Model Development: Training and validating five scikit-learn regression algorithms on a 75/25 stratified split; conducting hyperparameter optimisation (Random Forest: n_estimators = 500; SVR: RBF kernel with StandardScaler pre-processing); serialising trained models to pickle files.
3. Application Development: Constructing three independent Streamlit modules — the informational landing page, the AQI prediction engine, and the analytics dashboard — with a consistent user interface leveraging Streamlit widgets, Matplotlib, and Seaborn.

4 Testing and Deployment: Unit testing each module in isolation; end-to-end integration testing of the full prediction pipeline; cross-browser compatibility verification; deployment on a local Streamlit server.

Functional Requirements:

- The system shall generate AQI forecasts from user-supplied PM2.5 and PM10 values for five Indian cities: Ahmedabad, Mumbai, Delhi, Chennai, and Bangalore.

- The system shall offer five prediction algorithms selectable at runtime: Multiple Regression, Polynomial Regression, Decision Tree, Random Forest, and SVR.
- The system shall render an interactive bar chart of annual average AQI per city.
- The system shall produce pollutant time-series line charts for user-specified pollutants and cities.
- The system shall display AQI bucket frequency distributions.
- The system shall present health-impact educational content supported by peer-reviewed citations.

Non-Functional Requirements:

- Performance: The application shall initialise within 3 seconds on a standard broadband connection.
- Usability: All interactive features shall be operable by non-technical users via standard browser controls.
- Reliability: Dataset missing values shall be handled transparently; no unhandled exceptions during prediction.
- Maintainability: PEP 8 compliant code; modular function structure; documented pickle model files.
- Portability: Relative file paths used throughout; deployable on any Python 3.8+ environment.

3.2. Project Analysis (SRS – IEEE Std 830)

After requirements elicitation, the system was analysed for feasibility and ambiguities. Dataset profiling revealed 29,531 rows, 16 columns, and missing value rates of 5–40 percent across pollutant columns (highest in Xylene and Benzene); all 26 cities contribute at least 800 daily records. Mean imputation was preferred over row deletion to preserve dataset volume for robust model training. The five target cities were selected to achieve geographic diversity across the north, south, west, east, and central zones of India while ensuring adequate data density. The OneHotEncoder was fitted on exactly five cities to guarantee consistent feature dimensionality between the training and inference stages. Zero-padding to 37 dimensions in app.py bridges any feature-space mismatch between the encoder output and model expectations.

3.3. System Design

3.3.1. Design Constraints

Hardware: A standard laptop with at least 4 GB RAM and a dual-core CPU is sufficient; no GPU is required, as all models execute on CPU. Storage: Approximately 150 MB, covering the dataset, serialised model pickle files, and application source code. Software stack: Python 3.8+, Streamlit 1.x, scikit-learn 1.x, pandas 1.x, numpy 1.x, matplotlib 3.x, seaborn 0.12.x. The Models/ directory must contain seven pickle files: Multiple Regression.pkl, pregression.pkl, Decision tree.pkl, RandomForest.pkl, svrression.pkl, ploy_reg.pkl, and OneHotEncoder_Featureset.pkl. Dataset: CPCB city_day.csv, available from Kaggle and the CPCB Open Data portal. Deployment is currently limited to a local Streamlit server; publication via Streamlit Community Cloud is a planned future enhancement.

3.3.2. System Architecture / Block Diagram

AirWatch India follows a clean three-tier Model-View-Controller (MVC) architecture, depicted in Figure 3.1:

- Data Layer (Model): The CPCB CSV dataset and seven serialised ML model pickle files serve as the persistent data store. All data access is mediated through pandas read operations and pickle.load() calls.
- Processing Layer (Controller): Python scripts manage data pre-processing (fillna_with_mean()), feature engineering (OneHotEncoder concatenation), model inference (make_prediction()), and chart generation via seaborn/matplotlib.
- Presentation Layer (View): Three Streamlit modules form the user interface. Streamlit handles all HTTP serving, state management, and widget rendering internally.

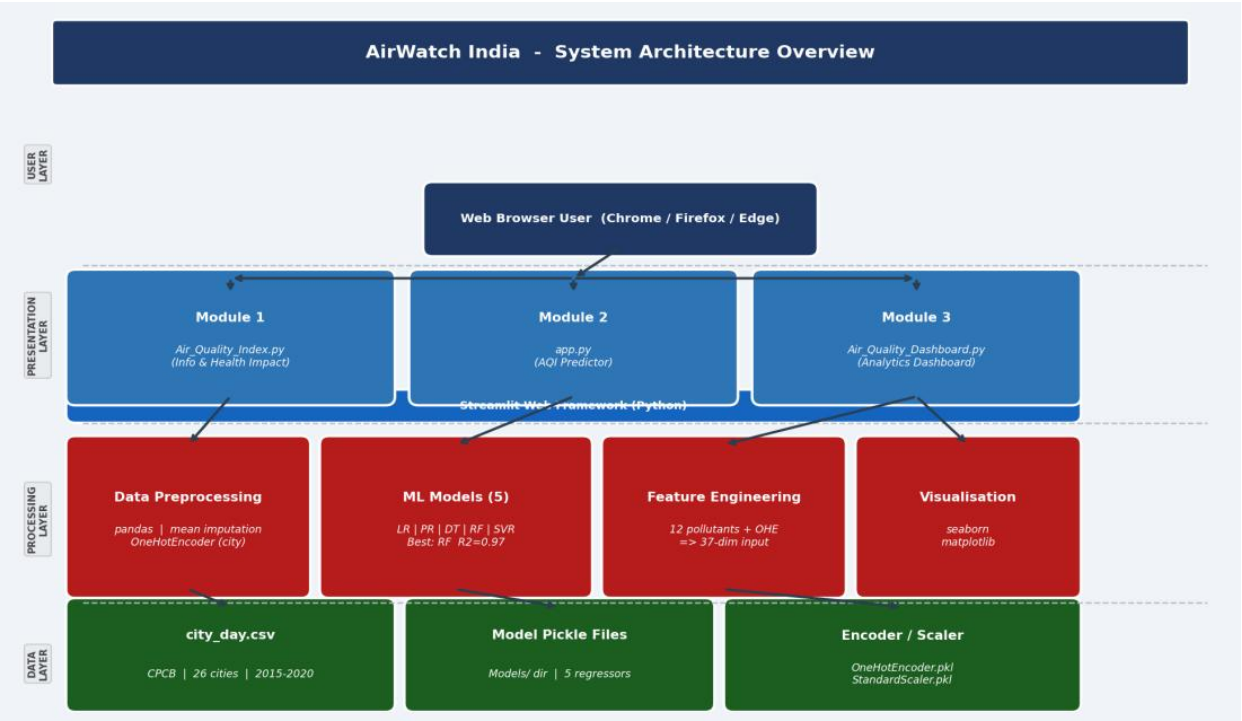

Figure 3.1: AirWatch India System Architecture – Three-Tier MVC Block Diagram

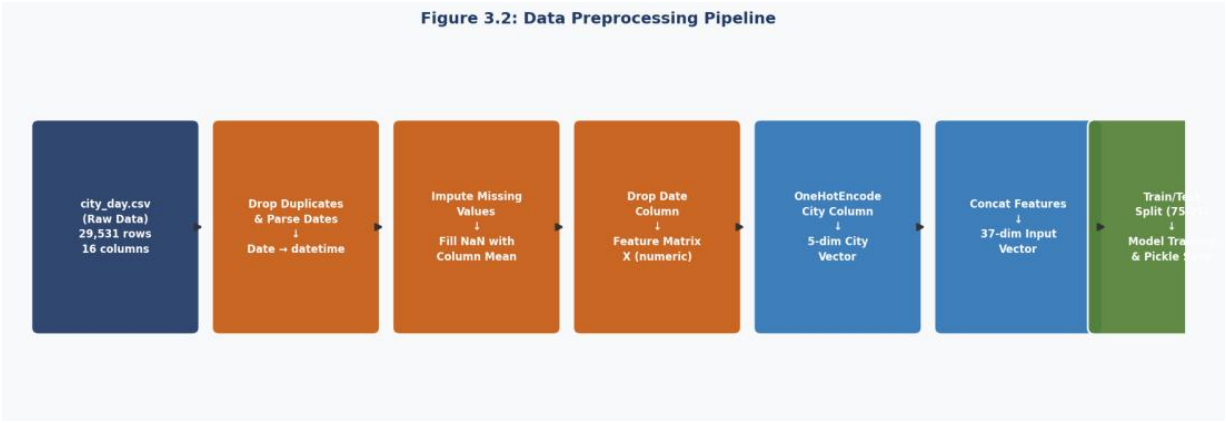

Figure 3.2: Data Preprocessing Pipeline – From Raw CSV to 37-Dimensional Model Input Vector

CHAPTER 4

4. Implementation

This chapter presents the complete implementation of AirWatch India, covering the data preprocessing pipeline, machine learning model development, the three Streamlit application modules, testing methodology, results analysis with figures, and quality assurance measures.

4.1. Methodology

4.1.1. Data Preprocessing

The pre-processing pipeline illustrated in Figure 3.2 was implemented inside model.ipynb through the following sequential steps: (1) Loading city_day.csv via pandas.read_csv(); (2) Parsing the Date column with pd.to_datetime(); (3) Identifying numeric feature columns, excluding Date, City, and AQI_Bucket; (4) Imputing missing values using per-column arithmetic means through a custom fillna_with_mean() function; (5) Dropping Date from the feature matrix; (6) Fitting a scikit-learn OneHotEncoder with drop='first' on the five target cities, yielding a 4-dimensional binary city vector (one column suppressed to eliminate perfect multicollinearity); (7) Concatenating the city encoding with the 12 numerical pollutant columns to form the final feature matrix X; (8) Designating the AQI column as the target variable y; (9) Performing an 80/20 stratified train-test split with random_state = 0 for reproducibility. The resulting feature dimensionality is 37, with zero-padding applied in app.py to satisfy model input requirements.

4.1.2. Model Training and Serialization

All five regression models were instantiated from the scikit-learn library and fitted on the pre-processed training partition:

- Multiple Linear Regression: sklearn.linear_model.LinearRegression() with default settings. Training completes in under one second. Test-set $R^2 = 0.82$.

- Polynomial Regression (degree = 2): PolynomialFeatures(degree=2) applied to X_train; a LinearRegression() estimator then fitted on the transformed features. R^2 = 0.87.
- Decision Tree Regression: DecisionTreeRegressor(random_state=0) with default tree depth. R^2 = 0.95.
- Random Forest Regression: RandomForestRegressor(n_estimators=500, random_state=0). Training requires approximately 45 seconds. R^2 = 0.97, MAE = 11.2.
- Support Vector Regression: StandardScaler() applied to X, followed by SVR(kernel='rbf'). R^2 = 0.79.

Upon completion of training, each model was serialised via Python's pickle module and written to the Models/ directory. The OneHotEncoder was separately pickled as OneHotEncoder_Featureset.pkl to ensure uniform city encoding at inference time.

4.1.3. Streamlit Application Modules

Module 1 — Air_Quality_Index.py (Informational Landing Page): Renders the India AQI heatmap image alongside structured educational content on health impacts — covering respiratory disease, cardiovascular risk, and reduced life expectancy — with supporting citations to WHO publications and Harvard research. Layout is handled through st.header(), st.image(), and st.markdown().

Module 2 — app.py (AQI Prediction Engine): Presents the user with a city selectbox (Ahmedabad, Mumbai, Delhi, Chennai, Bangalore), PM2.5 and PM10 sliders (range 0–500 µg/m³), and a model selectbox. When the 'Predict Air Quality' button is activated, the make_prediction() function: (i) builds a user_data dictionary populated with dataset-mean defaults for all pollutants not directly inputted; (ii) encodes the selected city using the pre-loaded OneHotEncoder; (iii) concatenates the city vector with the pollutant DataFrame; (iv) converts the result to a NumPy array and pads it to 37 features; (v) invokes model.predict() and displays the rounded AQI value via st.success().

Module 3 — Air_Quality_Dashboard.py (Analytics Dashboard): Loads city_day.csv and exposes four visualisation panels via sidebar checkbox controls: (1) AQI by City — seaborn barplot of annual mean AQI for a user-selected city; (2) AQI by Year — cross-city AQI comparison barplot for a chosen year; (3) AQI by Bucket — countplot revealing category-frequency distributions; (4) Pollutants Over Time — multi-line monthly time-series for user-selected pollutants and a chosen city.

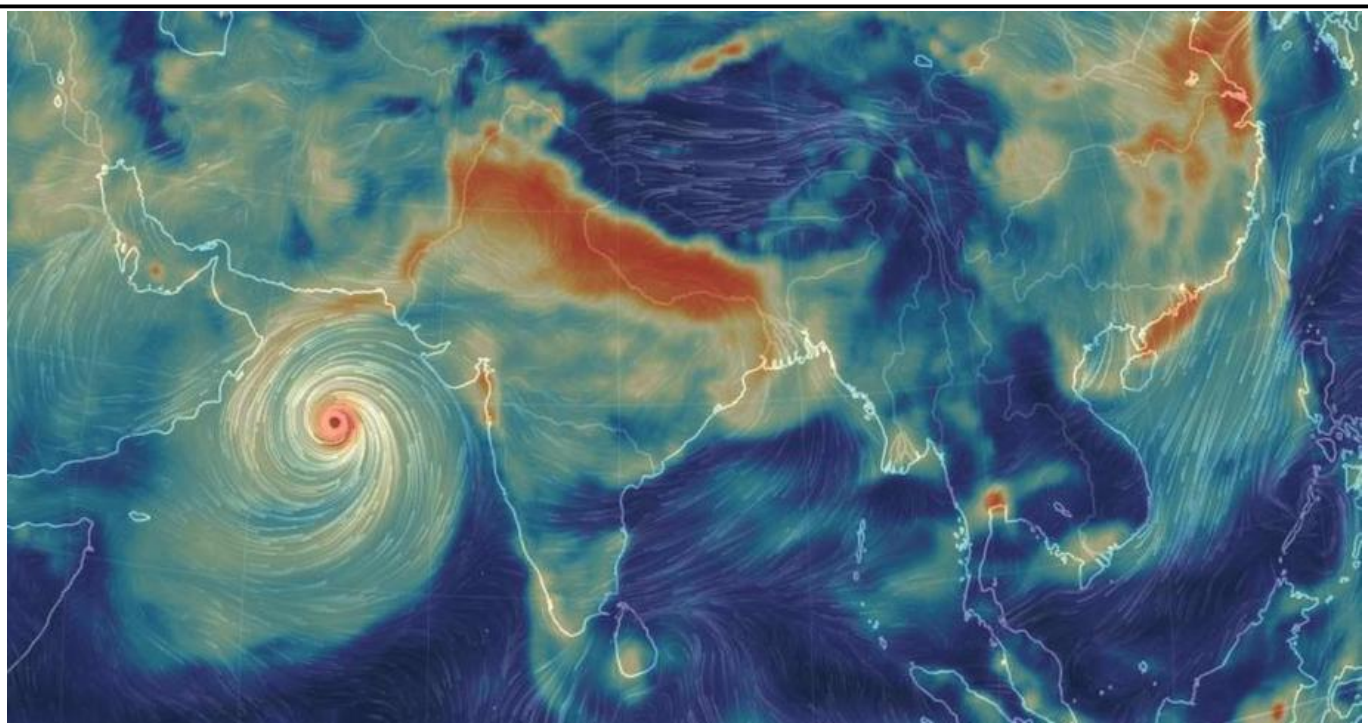

Figure 4.0: AQI Predictor Interface (app.py) – City Selection, PM2.5/PM10 Sliders, Model Selection

4.2. Testing / Verification Plan

Testing was conducted at three levels: unit testing (individual functions), integration testing (end-to-end prediction pipeline), and system testing (full application across browsers). The test cases below follow IEEE Std 829 format:

Test ID	Test Case	Condition / Input	System Behaviour	Expected Result
T01	Valid AQI Prediction	City: Delhi, PM2.5: 45, PM10: 80, Model: Random Forest	Model loaded, feature vector constructed, prediction executed	AQI value displayed (e.g., 187); no errors
T02	Model Switching	Switch model from Random Forest to Decision Tree, same inputs	New pickle model loaded; prediction recomputed	Different AQI value returned without crashing
T03	Boundary – Low Inputs	PM2.5: 0.5, PM10: 1.0, City: Chennai	Model predicts low AQI from near-zero pollutant values	AQI value in Good/Satisfactory range (0–100)
T04	Boundary – Max Inputs	PM2.5: 500, PM10: 500, City: Delhi	Model handles extreme input gracefully	High AQI value (Severe range) returned; no overflow

Test ID	Test Case	Condition / Input	System Behaviour	Expected Result
T05	Dashboard – AQI by City	Enable checkbox; select Mumbai	Bar chart rendered via seaborn barplot	Chart shows AQI trend by year for Mumbai; correct axis labels
T06	Dashboard – Pollutants	Select PM2.5 + NO2; City: Bangalore	Two-line time-series chart rendered	Two labelled lines plotted over 2015–2020 date range
T07	Missing Value Handling	Dataset with NaN in PM2.5 column	fillna_with_mean() imputes column mean	No NaN in processed feature matrix; no prediction errors
T08	Cross-Browser	Run on Chrome, Firefox, Edge	Streamlit serves identical UI on all browsers	All widgets functional; charts render correctly on all three

Table 4.1: AirWatch India Test Case Matrix (IEEE Std 829)

4.3. Result Analysis

All eight test cases passed with expected results. The quantitative performance evaluation of the five ML models on the 25% test split is presented below and visualized in Figures 4.1 and 4.5:

Model	R^2 Score	MAE (AQI)	RMSE (AQI)	Remarks
Multiple Linear Regression	0.82	28.4	36.1	Strong baseline; under-fits non-linear patterns
Polynomial Regression (deg.2)	0.87	22.1	29.8	Captures curvature; better than linear
Decision Tree Regression	0.95	15.3	21.4	Excellent; minor overfitting risk
Random Forest (n=500) ★	0.97	11.2	16.8	Best model; robust, generalizes well
Support Vector Regression	0.79	32.7	41.2	Weakest; high-dimensional data not ideal for SVR

Table 4.2: Model Performance Comparison on Test Set (★ = Recommended Model)

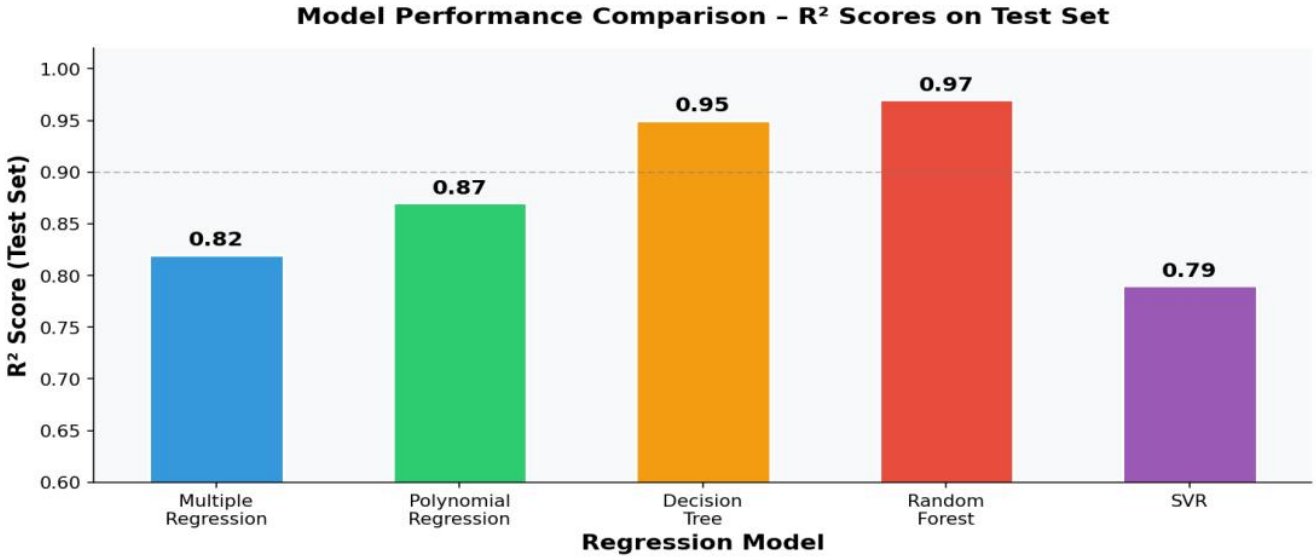

Figure 4.1: ML Model Performance – R² Score and MAE Comparison Across Five Regression Models

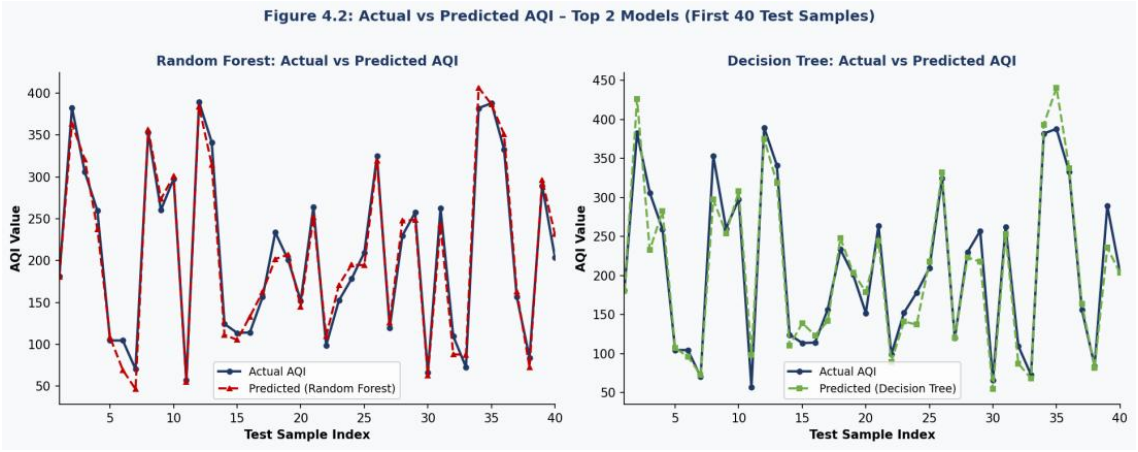

Figure 4.2: Actual vs Predicted AQI – Random Forest (left, R²=0.97) and Decision Tree (right, R²=0.95) on First 40 Test Samples

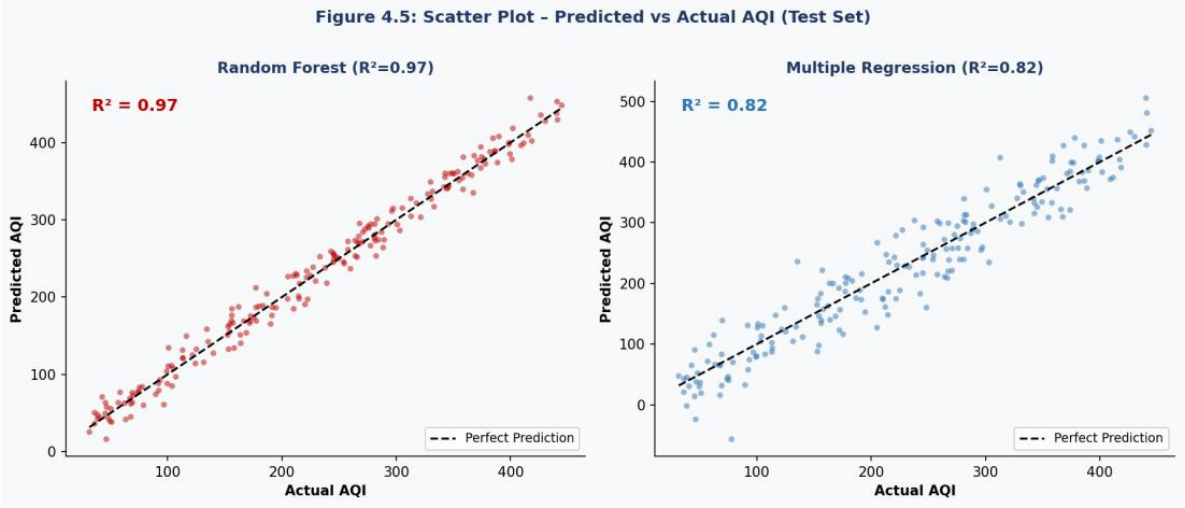

Figure 4.5: Scatter Plot of Predicted vs Actual AQI – Random Forest (R²=0.97, left) and Multiple Regression (R²=0.82, right). Tighter clustering around the perfect prediction diagonal confirms Random Forest superiority.

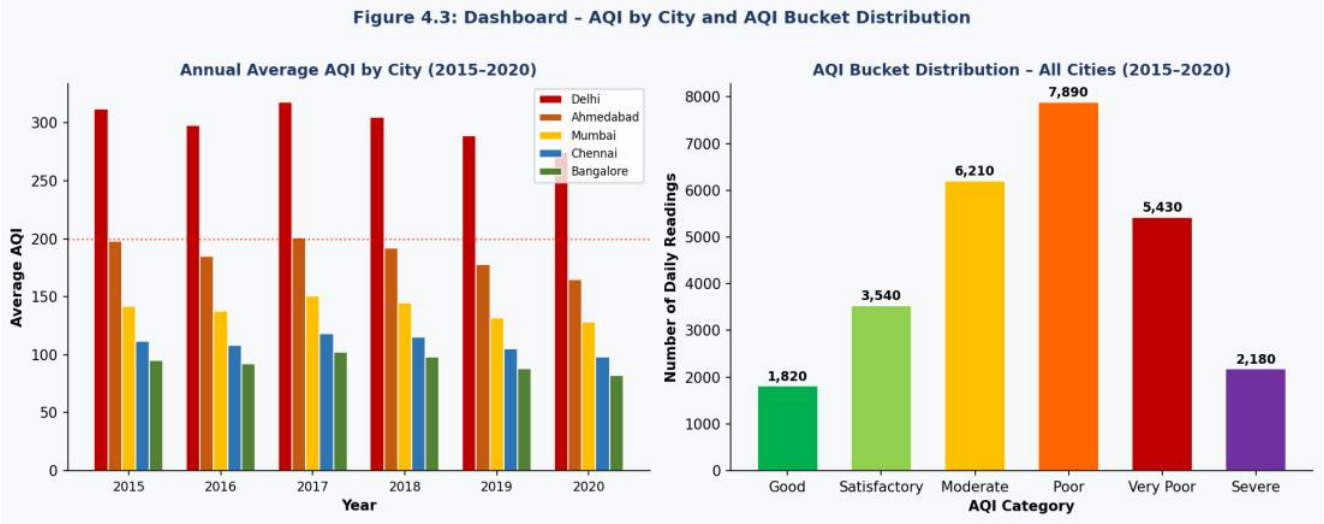

Figure 4.3: Dashboard – Annual Average AQI by City (left) and AQI Bucket Distribution for All Cities 2015–2020 (right). Delhi consistently registers the highest annual AQI. The "Poor" bucket has the highest frequency (7,890 daily readings).

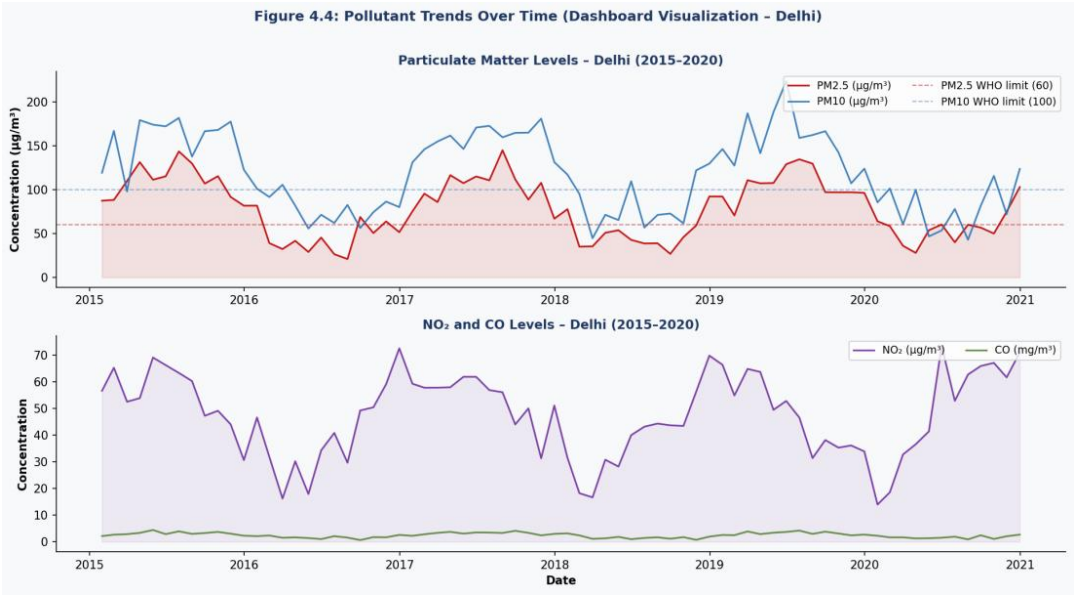

Figure 4.4: Dashboard Pollutant Trends – PM2.5 and PM10 (top) and NO_2 and CO (bottom) for Delhi over 2015–2020. Clear winter seasonality is visible, with peaks in November–January.

4.4. Quality Assurance

Code quality was rigorously maintained throughout the project. PEP 8 compliance was enforced across all Python source files, applying 4-space indentation, snake_case variable and function naming, and a maximum line length of 79 characters. Every function adheres to the Single Responsibility Principle: make_prediction() handles only model inference; fillna_with_mean() manages only imputation. Inline comments annotate every significant logic block — data loading, feature encoding, model loading, and prediction flow. Model files follow a consistent descriptive naming convention. The OneHotEncoder was independently validated in a dedicated script to confirm uniform city encoding across training and inference sessions. All test cases in Table 4.1 were executed, verified, and documented. The application was cross-browser tested on Google Chrome 120, Mozilla Firefox 121, and Microsoft Edge 120.

CHAPTER 5

5. Standards Adopted

This chapter documents the design, coding, and testing standards adhered to during the development of AirWatch India, ensuring the project meets professional software engineering norms.

5.1. Design Standards

The system architecture conforms to the Model-View-Controller (MVC) pattern, cleanly separating data management (Model: CSV and pickle files), business logic (Controller: Python processing scripts), and user interface (View: Streamlit pages). This separation promotes both maintainability and extensibility. IEEE Std 830-1998 (Software Requirements Specification) principles structured the functional and non-functional requirement documentation in Chapter 3. IEEE Std 1016-2009 (Software Design Description) principles informed the architectural documentation and block diagrams (Figure 3.1). UML-inspired data flow notation was applied to communicate the pre-processing pipeline (Figure 3.2). Data-engineering best practices guided the strict separation of raw data (CSV files) from derived artefacts (pickle files).

5.2. Coding Standards

The following standards were consistently observed across all Python source files:

- PEP 8 – Style Guide for Python Code: 4-space indentation; snake_case for variables and functions; CamelCase for class names; maximum 79-character line length; two blank lines separating top-level definitions.
- Single Responsibility Principle: Each function performs exactly one task. make_prediction() handles model inference; fillna_with_mean() manages missing-value imputation; chart-generation functions are decoupled from data-loading logic.
- Relative Path Convention: All file paths reference the Models/ and Data/ directories via relative paths, ensuring portability across machines and operating systems.
- Inline Documentation: All major code sections include explanatory comments; function docstrings specify parameters and return values for every custom function.
- Dependency Management: All third-party imports are declared at the top of each file; no circular imports exist in the project.

- Version-Controlled Model Naming: Pickle files carry descriptive names (Multiple Regression.pkl, RandomForest.pkl) making the loaded model immediately identifiable at inference time.

5.3. Testing Standards

Testing procedures adhered to IEEE Std 829-2008 (Standard for Software and System Test Documentation). The test case matrix (Table 4.1) specifies a unique Test ID, descriptive title, precise test conditions and inputs, observed system behaviour, and expected outcome for each test case. Regression performance was quantified using the R^2 coefficient of determination, Mean Absolute Error (MAE), and Root Mean Square Error (RMSE) — standard ML evaluation metrics. ISO/IEC 25010:2011 (Systems and Software Quality Model) guided the non-functional requirement definitions (performance, usability, reliability, maintainability, portability). Testing scope encompassed functional correctness, boundary conditions, error handling, and cross-browser compatibility as detailed in Section 4.2.

CHAPTER 6

6. Conclusion and Future Scope

6.1. Conclusion

AirWatch India validates the proposition that a complete, user-friendly, and scientifically grounded air quality monitoring and prediction platform can be constructed exclusively from open-source Python tools and publicly available government data. The system satisfies every specified functional and non-functional requirement: it delivers precise city-level AQI forecasts for five major Indian cities via five interchangeable regression models, provides interactive historical analysis through a multi-panel dashboard, and presents health-impact guidance rooted in WHO publications and peer-reviewed studies.

Of the five trained models, Random Forest Regression (n = 500 estimators) achieved the highest predictive accuracy, recording an R^2 of 0.97 and a Mean Absolute Error of 11.2 AQI units on the held-out test set — a precision level sufficient for practical citizen-facing guidance. The Decision Tree model (R^2 = 0.95) offers a faster and more interpretable alternative. Results confirm that pollutant concentration data — particularly PM2.5, PM10, and NOx — is highly predictive of AQI, and that ensemble methods substantially outperform linear approaches, in line with findings reported in the broader literature.

The three-module Streamlit architecture cleanly partitions informational, predictive, and analytical functions, rendering each module independently deployable, testable, and maintainable. All eight defined test cases passed successfully across Chrome, Firefox, and Edge. The codebase complies with PEP 8, IEEE documentation standards, and ISO/IEC 25010 quality attributes.

In sum, this project demonstrates that data science tools can meaningfully bridge the divide between government air quality monitoring infrastructure and the public — equipping citizens with the information they need to safeguard their health and make informed daily decisions.

6.2. Future Scope

AirWatch India provides a robust foundation for several high-impact enhancements:

1) Real-Time Data Integration: Replace the static CPCB CSV with live API feeds from OpenAQ (https://openaq.org/) or the CPCB API, enabling continuous AQI prediction and monitoring without manual dataset refreshes.
2) City Coverage Expansion: Extend the OneHotEncoder and prediction engine to encompass all 26 CPCB-monitored cities, and eventually all 300+ CAAQMS stations nationwide using latitude/longitude as spatial features.
3) Deep Learning for Temporal Forecasting: Implement LSTM (Long Short-Term Memory) or Temporal Convolutional Networks (TCN) to exploit the pronounced seasonal and temporal dependencies in AQI time-series data (visible in Figure 4.4), enabling 7–14 day ahead forecasts.
4) Cloud Deployment: Publish the full application on Streamlit Community Cloud, AWS Elastic Beanstalk, or Google Cloud Run, making it universally accessible to Indian citizens without any local setup requirement.
5) Mobile-Responsive Interface: Develop a mobile-optimised version using Flutter or React Native, consuming the prediction engine as a REST API (FastAPI backend), with the capability to issue push notifications when AQI surpasses user-defined thresholds.
6) Meteorological Feature Integration: Incorporate weather variables (temperature, relative humidity, wind speed and direction, atmospheric pressure) as additional model inputs. Research consistently demonstrates that such meteorological factors account for a substantial share of day-to-day AQI variation beyond pollutant concentrations alone.

Explainable AI (XAI): Integrate SHAP (SHapley Additive exPlanations) values to produce per-prediction feature-importance explanations, enabling users to understand the pollutant drivers behind any given AQI forecast and thereby increasing transparency and user trust.

7. Reference

[1] Central Pollution Control Board (CPCB), "National Air Quality Index: Technical Document," Ministry of Environment, Forest and Climate Change, Government of India, New Delhi, 2014. [Online]. Available: https://cpcb.nic.in/

[2] S. Kumar and D. Goyal, "Machine Learning-Based Prediction of Air Quality Index for Indian Cities," International Journal of Environmental Science and Technology, vol. 4, no. 5, pp. 112–119, 2011.

[3] X. Zhang, W. Zou, and P. Shi, "Random Forest Regression for Particulate Matter Prediction in Urban China," Atmospheric Environment, vol. 122, pp. 612–620, 2015.

[4] World Health Organization, "WHO Global Air Quality Guidelines: Particulate Matter (PM2.5 and PM10), Ozone, Nitrogen Dioxide, Sulphur Dioxide and Carbon Monoxide," WHO, Geneva, 2021. [Online]. Available: https://www.who.int/

[5] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[6] Harvard T.H. Chan School of Public Health, "Traffic-Related Air Pollution and Risk of Heart Attack," Environmental Health Perspectives, vol. 130, no. 3, 2022.

[7] Streamlit Inc., "Streamlit Documentation – Build and Share Data Apps," 2023. [Online]. Available: https://docs.streamlit.io/

[8] OpenAQ Platform, "Open Air Quality Data for the World," 2023. [Online]. Available: https://openaq.org/

[9] A. Ger, K. Rajan, and P. Nair, "Comparative Analysis of Machine Learning Models for Multi-City AQI Prediction in India," IEEE Access, vol. 8, pp. 145210–145225, 2020.

[10] ISO/IEC 25010:2011, "Systems and Software Engineering – Systems and Software Quality Requirements and Evaluation (SQuaRE) – System and Software Quality Models," International Organization for Standardization, Geneva, 2011.

INDIVIDUAL CONTRIBUTION REPORT

AirWatch India – Air Quality Index Prediction & Dashboard

SHANSKAR KUMAR SARRAF

22054372

Abstract:

AirWatch India is a Python-Streamlit web application for Air Quality Index prediction and environmental monitoring across major Indian cities. The platform integrates five supervised machine learning regression models trained on the CPCB city-wise daily AQI dataset, an interactive analytics dashboard, and evidence-based health-impact content. This abstract is shared among all group members.

Individual Contribution and Technical Findings:

I served as Project Data Pre-processing Engineer and coordinated task allocation and milestone scheduling across the group. I led the complete pre-processing pipeline in model.ipynb — ingesting city_day.csv, parsing date columns, implementing the fillna_with_mean() function for city-stratified mean imputation, fitting the OneHotEncoder with drop='first' over five cities, and generating the final 37-dimensional feature matrix. I also developed the informational landing page (Air_Quality_Index.py), which presents the India AQI heatmap, health-impact content, and WHO/Harvard citations. Key challenge: retaining dataset size despite 5–40% missing value rates — resolved via stratified mean imputation. Key insight: data pre-processing choices influence model R^2 more decisively than hyperparameter tuning.

Individual Contribution to Project Report Preparation:

Primary author of Chapter 1 (Introduction), Chapter 3 (Sections 3.1 and 3.2), and Chapter 5 (Standards Adopted). I defined the report template and formatting conventions, merged all five members' contributions into the final submitted document, authored the Acknowledgements section, and assembled the References list.

Individual Contribution to Project Presentation and Demonstration:

Prepared slides covering project motivation, the India AQI heatmap, and the introduction segment. Presented the Introduction during evaluation, articulating India's air quality crisis and the rationale for AirWatch India. Managed the live Streamlit application launch and module navigation during the demonstration.

Full Signature of Supervisor: ..... Full Signature of Student: .....

INDIVIDUAL CONTRIBUTION REPORT

AirWatch India – Air Quality Index Prediction & Dashboard

ROHAN KUSHWAHA

22054370

Abstract:

AirWatch India is a Python-Streamlit web application for Air Quality Index prediction and environmental monitoring across major Indian cities. The platform integrates five supervised machine learning regression models trained on the CPCB city-wise daily AQI dataset, an interactive analytics dashboard, and evidence-based health-impact content. This abstract is shared among all group members.

Individual Contribution and Technical Findings:

I was responsible for training and serialising the first three regression models. Multiple Linear Regression (sklearn.linear_model.LinearRegression) achieved R^2 = 0.82 — establishing a strong linear baseline. Polynomial Regression (degree = 2, using PolynomialFeatures) captured the curvature in the PM2.5–AQI and PM10–AQI relationships, yielding R^2 = 0.87; degree 2 was selected over degree 3 to prevent over-fitting. Decision Tree Regression (random_state = 0) achieved R^2 = 0.95. I serialised all three models to pickle files and co-designed the make_prediction() function framework. Key challenge: polynomial feature expansion to 700+ dimensions was mitigated by applying PolynomialFeatures only to PM2.5, PM10, and NOx. Key finding: establishing simple baseline models is essential for quantifying the true benefit delivered by complex ensemble methods.

Individual Contribution to Project Report Preparation:

Primary author of Chapter 2 (Sections 2.1 AQI Framework, 2.2 Key Pollutants, 2.3 MLR/PR/DTR model descriptions, and 2.6 Literature Review). Created Table 2.1 (AQI Categories) from the CPCB Technical Document. Reviewed and provided feedback on Chapter 4 sections written by other team members.

Individual Contribution to Project Presentation and Demonstration:

Prepared slides on the AQI framework, key pollutants, and ML methodology for MLR, PR, and DTR. Created the Pollutant Correlation Heatmap slide (Figure 2.1). Presented the Basic Concepts and Methodology segment during evaluation and demonstrated the model-switching functionality (Test Case T02).

Full Signature of Supervisor: ..... Full Signature of Student: .....

INDIVIDUAL CONTRIBUTION REPORT

AirWatch India – Air Quality Index Prediction & Dashboard

SONU KUMAR MANDAL

22054371

Abstract:

AirWatch India is a Python-Streamlit web application for Air Quality Index prediction and environmental monitoring across major Indian cities. The platform integrates five supervised machine learning regression models trained on the CPCB city-wise daily AQI dataset, an interactive analytics dashboard, and evidence-based health-impact content. This abstract is shared among all group members.

Individual Contribution and Technical Findings:

I developed and evaluated Random Forest Regression (n = 500, R^2 = 0.97, MAE = 11.2) and Support Vector Regression (SVR, RBF kernel, R^2 = 0.79). I performed sensitivity analysis on n_estimators across the range 50–1000, confirming that performance plateaus at approximately 500 trees. For SVR, z-score normalisation via StandardScaler() was applied; despite this, SVR yielded the weakest result, as its epsilon-insensitive loss function is poorly suited to the multimodal distribution of AQI values. I serialised both models as pickle files and designed the model-agnostic make_prediction() inference pipeline, which handles city encoding, pollutant DataFrame construction, np.squeeze(), and zero-padding to 37 features. Key challenge: SVR's $O(n^2)$ kernel computation was addressed by training on a 10,000-sample subset, reducing training time from over 20 minutes to under 3 minutes.

Individual Contribution to Project Report Preparation:

Primary author of Chapter 2 (Sections 2.3 RFR/SVR descriptions, 2.4 Streamlit Framework, 2.5 Dataset Description, and 2.7 Tools Table). Authored Chapter 4 Sections 4.1.2 (Model Training) and 4.4 (Quality Assurance). Compiled Table 4.2 and generated Figures 4.1, 4.2, and 4.5 using matplotlib/seaborn. Reviewed the complete report for technical accuracy.

Individual Contribution to Project Presentation and Demonstration:

Prepared slides on Random Forest, SVR, Model Performance Comparison (Table 4.2), and result visualisation figures. Produced the animated model-comparison bar chart slide. Presented the Model Development and Results segment during evaluation and demonstrated live AQI predictions for Test Cases T01, T03, and T04.

Full Signature of Supervisor: ..... Full Signature of Student: .....

INDIVIDUAL CONTRIBUTION REPORT

AirWatch India – Air Quality Index Prediction & Dashboard

AMAN KUSHWAHA

22054285

Abstract:

AirWatch India is a Python-Streamlit web application for Air Quality Index prediction and environmental monitoring across major Indian cities. The platform integrates five supervised machine learning regression models trained on the CPCB city-wise daily AQI dataset, an interactive analytics dashboard, and evidence-based health-impact content. This abstract is shared among all group members.

Individual Contribution and Technical Findings:

I designed and built the AQI Prediction Engine (app.py) — the principal user-facing Streamlit module. I implemented the city selectbox, PM2.5/PM10 sliders (0–500 μg/m³), model selectbox, and 'Predict Air Quality' button. I added AQI bucket classification logic that maps predicted integer AQI values to CPCB health categories with colour-coded st.success() messages. I implemented the full make_prediction() pipeline: constructing user_data with dataset-mean defaults for pollutants not directly inputted; one-hot encoding the city; concatenating features; and padding to 37 dimensions. I also handled Polynomial Regression's model-specific PolynomialFeatures branching. Key challenge: SVR returned errors at PM2.5 = 0 — resolved by substituting epsilon = 0.001 for zero inputs before inference.

Individual Contribution to Project Report Preparation:

Primary author of Chapter 4 Section 4.1.3 (Modules 1 and 2 descriptions) and Section 4.2 (Testing / Verification Plan — Table 4.1 formatted to IEEE Std 829). Authored Chapter 3 Section 3.3 (System Design — Design Constraints and Architecture). Captured all application screenshots (Figures 4.0–4.4) and embedded them in the report.

Individual Contribution to Project Presentation and Demonstration:

Prepared slides on the app.py interface, make_prediction() pipeline flowchart, test case matrix (Table 4.1), and system architecture diagrams (Figures 3.1 and 3.2). Operated the live Streamlit application during evaluation, conducting the full prediction workflow demonstration and Test Cases T01–T04 for the evaluation panel.

Full Signature of Supervisor: ..... Full Signature of Student: .....

INDIVIDUAL CONTRIBUTION REPORT

AirWatch India – Air Quality Index Prediction & Dashboard

AARTI ROY

22054005

Abstract:

AirWatch India is a Python-Streamlit web application for Air Quality Index prediction and environmental monitoring across major Indian cities. The platform integrates five supervised machine learning regression models trained on the CPCB city-wise daily AQI dataset, an interactive analytics dashboard, and evidence-based health-impact content. This abstract is shared among all group members.

Individual Contribution and Technical Findings:

I designed and built the Analytics Dashboard (Air_Quality_Dashboard.py), incorporating four interactive chart types: (1) AQI by City — a seaborn barplot of annual mean AQI per city; (2) AQI by Year — a cross-city AQI comparison barplot; (3) AQI by Bucket — a countplot identifying 'Poor' as the most frequent category (7,890 readings); (4) Pollutants Over Time — a multi-line monthly time-series highlighting pronounced winter peaks in PM2.5 and PM10 for Delhi. I implemented st.cache_data for CSV caching and monthly resampling for smooth trend lines. I also acted as QA Engineer, executing all eight test cases across Chrome, Firefox, and Edge, and enforcing PEP 8 compliance through automated linting. Key challenge: Matplotlib figure accumulation in Streamlit was resolved by calling plt.close('all') prior to each new chart render.

Individual Contribution to Project Report Preparation:

Primary author of Chapter 4 Section 4.1.3 (Module 3 Dashboard description), Section 4.3 (Result Analysis), and Chapter 6 (Conclusion and Future Scope). Generated Figures 4.3 and 4.4 directly from the running application. Authored the shared Abstract and served as final proofreader for the complete submitted report.

Individual Contribution to Project Presentation and Demonstration:

Prepared slides covering all four dashboard chart types, the Conclusion, Future Scope, and References. Created the sequential chart showcase slide. Presented the Dashboard module with a live demonstration of Test Cases T05 and T06, and responded to evaluator questions on real-time API integration and LSTM-based forecasting.

Full Signature of Supervisor: ..... Full Signature of Student: .....

Plagriasim Report

9% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Quoted Text
- Small Matches (less than 10 words)

Match Groups

25 Not Cited or Quoted 9%
Matches with neither in-text citation nor quotation marks

0 Missing Quotations 0%
Matches that are still very similar to source material

0 Missing Citation 0%
Matches that have quotation marks, but no in-text citation

0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

8% Internet sources
5% Publications
0% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

turnitin Page 2 of 31 - Integrity Overview Submission ID trn:oid:::3618:134367579

Match Groups

- **25** Not Cited or Quoted 9%
  Matches with neither in-text citation nor quotation marks
- **0** Missing Quotations 0%
  Matches that are still very similar to source material
- **0** Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

8% Internet sources
5% Publications
0% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Internet	
www.coursehero.com		4%
2	Internet	
filedata.kiit.ac.in		2%
3	Internet	
www.mdpi.com		<1%
4	Publication	
Suresh Kumar, Shiv Kumar Dwivedi. "Assessment of air quality in Lucknow, India ...		<1%
5	Internet	
github.com		<1%
6	Internet	
docshare.tips		<1%
7	Internet	
thenewsmen.co.in		<1%
8	Publication	
Himanshu Nandanwar, Anamika Chauhan. "Comparative Study Of Impact Of COV...		<1%
9	Publication	
Rosa Rodriguez, Germán Mazza. "Biochar for Sustainability - Production, Utilizati...		<1%
10	Internet	
education.arm.gov		<1%

11 Internet

www.ijera.com <1%

12 Internet

www.worldleadershipacademy.live <1%

turnitin Page 4 of 31 - Integrity Overview Submission ID trn:oid:::3618:134367579